

MEMORIA PRÁCTICA 2

Avance Rápido (H_AR) y Backtraking (L_Ba)

Cuenta: G1_07

Alumnas:

María Ballester Martínez (m.ballestermartinez@um.es) 48853989B

Lucia Olmos Martínez (lucia.olmosm@um.es) 49339235A

1.	Lista de problemas resueltos.....	2
a.	H_AR.....	2
b.	L_Ba	3
2.	AVANCE RAPIDO.....	4
a.	Pseudocódigo.....	4
b.	Código	10
c.	Número de envío	13
d.	Estudio teórico.....	13
e.	Estudio experimental	14
f.	Generador de casos	14
g.	Contraste de estudio teórico y experimental.....	14
3.	BACKTRAKING	15
a.	Justificación del tipo de árbol.....	15
b.	Pseudocódigo.....	15
c.	Código	22
d.	Numero de envío	26
e.	Estudio teórico.....	26
f.	Estudio experimental	27
g.	Generador de casos	28
h.	Contraste de estudio teórico y experimental.....	29
4.	Conclusiones.....	30

1. Lista de problemas resueltos

a. H_AR

Los dos fallos eran por lo mismo, hasta que no nos dimos cuenta de que error cometíamos no nos lo acepto.

Nosotras ordenamos las tareas de menor a mayor tiempo para cada máquina. Y el array "indices" nos indica por dónde va cada máquina, es decir, indica la tarea por la que va en su orden, la casilla.

Si la tarea en la que se encuentra no está libre ya, si ya se ha usado, debe avanzar hasta la siguiente libre.

Pero el error daba porque en vez de hacer el while para avanzar en la tarea:

```
while(indices[j] < N && candidatos[tiempos[j][indices[j]].tarea-1]){
    indices[j]++;
}
if(!candidatos[tiempos[j][indices[j]].tarea-1]){
    int fac = esFactible(tiempoMaq,indices,j,maq,tiempoTarea,tiempos);
    if(fac == 1){
        tiempoTarea = tiempos[j][indices[j]].tiempo;
        maq = j;
        tarea = tiempos[j][indices[j]].tarea-1;
    } else if(fac == 2){
        if(tiempoMaq[j]<=tiempoMaq[maq]){
            tiempoTarea = tiempos[j][indices[j]].tiempo;
            maq = j;
            tarea = tiempos[j][indices[j]].tarea-1;
        }
    }
}
```

Había un else y el error seguramente es que simplemente avanzaba a la siguiente tarea y no avanzaba hasta encontrar la siguiente no usada.

```
else {
    indices[j]++;
}
```

304	2024/05/02 10:06		G1 G1 07	H_AR	C++	0 Wrong Answer	final
-----	------------------	--	----------	------	-----	----------------	-------

419	2024/05/07 09:16		G1 G1 07	H_AR	C++	0 Wrong Answer	final
-----	------------------	--	----------	------	-----	----------------	-------

En este tercer y cuarto envío está bien, pero con muchos comentarios de cout que hemos usado para encontrar los errores o con código que no es necesario.

420	2024/05/07 09:52		G1 G1 07	H_AR	C++	3 Accepted	final
-----	------------------	--	----------	------	-----	------------	-------

421	2024/05/07 10:02		G1 G1 07	H_AR	C++	3 Accepted	final
-----	------------------	--	----------	------	-----	------------	-------

Versión final del código

438	2024/05/07 17:15		G1 G1 07	H_AR	C++	3 Accepted	final
-----	------------------	--	----------	------	-----	------------	-------

b. L_Ba

El error que se encontraba en el primer intento de envío se debía a que tardaba demasiado tiempo en ejecutarse porque se realizaba siempre la comprobación de si se tenía ya la solución.

```
if(solucion(nivel,sol,empleados)){  
    fin = true;  
}
```

430	2024/05/08 23:50	G1 G1 07	L_Ba	C++	1 Time Limit Exceeded	final
-----	------------------	----------	------	-----	-----------------------	-------

La solución fue añadir otra comprobación para que no se compruebe todas las veces y solo cuando exista la posibilidad real de que se haya encontrado la solución, es decir, lo comprobará cuando se haya llegado al último nivel del árbol:

```
if(nivel==m-1 && solucion(nivel,sol,empleados)){  
    fin = true;  
}
```

Volvimos a probar y lo aceptó, pero teníamos bastantes comentarios innecesarios, por lo que volvimos a enviarlo sin ellos.

431	2024/05/08 23:53	G1 G1 07	L_Ba	C++	3 Accepted	final
-----	------------------	----------	------	-----	------------	-------

Versión final del código

434	2024/05/09 00:03	G1 G1 07	L_Ba	C++	3 Accepted	final
-----	------------------	----------	------	-----	------------	-------

2. AVANCE RAPIDO

a. Pseudocódigo

Hemos creado dos estructuras de datos nuevas. Una representa la Tarea, con su número y el tiempo que tarda. La otra representa la forma de la Solución, conformada por el número de tarea y el número de la máquina en la que se ha elegido realizarla.

// Definición de estructuras

```
struct Tarea {  
    Entero tarea  
    Entero tiempo  
}  
  
struct Solucion {  
    Entero tarea  
    Entero maquina  
}
```

Función necesaria para poder usar la función “sort” del tipo de datos “vector”. Establece el comparador para saber cómo ordenar el vector, es decir, va comprobando que primero se encuentre la tarea más pequeña de las dos que se le pasan.

// Función para comparar tareas basadas en el tiempo

```
función comparador(Referencia constante Tarea a, Referencia constante Tarea b) → Booleano {  
    devolver a.tiempo < b.tiempo  
}
```

Función que comprueba si es factible escoger la opción que se le pasa, es decir, si se trata de una opción mejor de alguna manera que la opción que ya tenemos escogida.

Si tenemos 3 máquinas, encontramos lo siguiente en el vector tiempoMaq:

Máquina 0	Máquina 1	Máquina 2
12	11	6

Comprobación 1: Si el tiempo total de la máquina que es posible que vayamos a elegir más el tiempo de la tarea a elegir es mayor que el ya seleccionado, devuelve un 1.

Comprobación 2: Si el tiempo total de la máquina que es posible que vayamos a elegir más el tiempo de la tarea a elegir es igual que el ya seleccionado, devuelve un 2.

En otro caso, devuelve un 0.

// Función para determinar si una tarea es factible en una máquina dada

función esFactible(vector de enteros tiempoMaq, vector de enteros índices, entero j, entero maq, flotante tiempoTarea, vector de vector de Tarea tiempos) → entero {

 entero fac = 0

 si (tiempoMaq[j] + tiempos[j][índices[j]].tiempo < tiempoMaq[maq] + tiempoTarea)

 fac = 1

 sino si (tiempoMaq[j] + tiempos[j][índices[j]].tiempo == tiempoMaq[maq] + tiempoTarea)

 fac = 2

 devolver fac

}

Esta función tiene el algoritmo de avance rápido que llevará a cabo la elección de las tareas y máquinas.

- El vector “tiempoMaq(M)” contendrá el tiempo que lleva realizado cada máquina en cada momento según las tareas que se haya decidido realizar en cada una de ellas. Su tamaño es el del número de máquinas que haya.
- El vector “candidatos(N)” contendrá si cada una de las tareas ha sido elegida ya o no. Su tamaño es el del número de tareas que haya.
- El vector “solución(N)” contendrá la tarea que se ha decidido realizar en cada máquina, en el orden en el que se elige. Usa la estructura de datos “Solución”. Su tamaño es el del número de tareas que haya, ya que se tendrá que elegir todas las tareas.
- El vector “indices(M)” indica por dónde vamos en la “fila” de cada máquina, ya que no vamos avanzando a la vez en todas las máquinas. Su tamaño es el del número de tareas. Podemos encontrar el siguiente caso:

Máquina 0	Máquina 1	Máquina 2
Hueco 3	Hueco 2	Hueco 4

Se inicializan a 0 y falso los vectores correspondientes.

Después encontramos el bucle principal, que se ejecutará la cantidad de veces correspondiente al número de tareas que haya y será el que vaya eligiendo las tareas a realizar y las máquinas en las que se realizarán.

- tiempoTarea contendrá el tiempo que tarda en realizarse la tarea en la máquina que hemos escogido de momento.
- maq será la máquina escogida temporalmente.
- tarea tendrá el valor de i en cada iteración de este bucle.

Nos encontramos otro bucle que se ejecutará, en cada iteración del que lo contiene, la cantidad de veces correspondiente al número de máquinas que haya.

Se actualiza el hueco de la máquina correspondiente en el vector “indices” para avanzar hasta la siguiente tarea no elegida todavía.

Tras esto, comprobamos que el candidato actual de la máquina no se haya usado todavía.

Si no se ha usado todavía, se llama a la función esFactible y tratamos cada caso:

- Si la función ha devuelto 1, elegimos temporalmente la máquina y tarea actual.
- Si la función ha devuelto 2, comprobamos cuál es la máquina a la que se le añadirá menos tiempo y establecemos los valores temporales en base a esto.
- Si la función ha devuelto 0, continuamos con los valores temporales que ya teníamos guardados.

Cuando termine el segundo bucle, elegimos lo guardado temporalmente como parte de la respuesta.

Una vez termina el bucle principal, imprimimos los resultados.

// Función para asignar tareas a máquinas de manera eficiente

procedimiento avanceRapido(vector de vector de Tarea tiempos, entero N, entero M) {

 // Vector para almacenar el tiempo total actual de las máquinas

 vector de enteros tiempoMaq(M)

 // Vector booleano para marcar las tareas usadas

 vector de booleanos candidatos(N)

 // Vector para guardar la solución

 vector de Solucion solucion(N)

 // Vector para llevar los índices de cada máquina

 vector de enteros indices(M)

 // Inicialización de vectores

 para cada i en [0, M)

 indices[i] = 0

 tiempoMaq[i] = 0

 para cada i en [0, N)

 candidatos[i] = Falso

 // Iteración sobre las tareas

 para cada i en [0, N)

 tiempoTarea = INFINITO

 entero maq = 0

 entero tarea = i

 para cada j en [0, M)

 mientras (indices[j] < N y candidatos[tiempos[j]][indices[j]].tarea - 1))

 indices[j]++

 si (!candidatos[tiempos[j]][indices[j]].tarea - 1))

 entero fac = esFactible(tiempoMaq, indices, j, maq, tiempoTarea,
 tiempos)

 si (fac == 1)

 tiempoTarea = tiempos[j][indices[j]].tiempo


```

    maq = j
    tarea = tiempos[j][indices[j]].tarea - 1
sino si (fac == 2) Entonces
    si (tiempoMaq[j] <= tiempoMaq[maq]) entonces
        tiempoTarea = tiempos[j][indices[j]].tiempo
        maq = j
        tarea = tiempos[j][indices[j]].tarea - 1

```

```

candidatos[tarea] = Verdadero
tiempoMaq[maq] += tiempoTarea
Solucion sol
sol.maquina = maq
sol.tarea = tarea
solucion[i] = sol
indices[maq]++

```

// Cálculo del tiempo total

```

entero tiempoTotal = -1
para cada i en [0, M)
    si (tiempoMaq[i] > tiempoTotal)
        tiempoTotal = tiempoMaq[i]

```

```

escribir tiempoTotal
para cada i en [0, N)
    escribir solucion[i].tarea + 1
para cada i en [0, N)
    escribir solucion[i].maquina + 1

```

```

}

```

El main lee el número de casos, de tareas y máquinas por la entrada además de los tiempos.

- La matriz formada por vectores “tiempo” guarda los tiempos que tarda cada tarea en las máquinas en orden (de tarea).

Una vez la tabla está completa, ordena los tiempos de cada máquina de menor a mayor.

Llama a “avanceRapido”.

// Función principal

función Principal() → Entero {

 // Variables

 entero P, N, M // P → número de casos; N → número de tareas; M → número de máquinas

 leer P

 mientras (P > 0) hacer

 leer N, M

 // Vector para almacenar los tiempos de las tareas en las máquinas

 vector de vector de Tarea tiempos(M, vector de Tarea(N))

 para cada i en [0, N)

 para cada j en [0, M)

 leer tiempos[j][i].tiempo

 tiempos[j][i].tarea = i + 1

 para cada i en [0, M)

 ordenar tiempos[i] usando comparador

 avanceRapido(tiempos, N, M)

 P--

 }

b. Código

```
#include <iostream>
#include <vector>
#include <numeric>
#include <algorithm>
#include <math.h>

using namespace std;

struct Tarea {
    int tarea;
    int tiempo;
};

struct Solucion {
    int tarea;
    int maquina;
};

bool comparador(const Tarea& a, const Tarea& b) { //Criterio de
organizacion de vector
    return a.tiempo < b.tiempo;
}

int esFactible(vector<int> tiempoMaq, vector<int> indices, int j, int
maq, float tiempoTarea, vector<vector<Tarea>>tiempos){
    int fac = 0;
    if(tiempoMaq[j]+tiempos[j][indices[j]].tiempo <
tiempoMaq[maq]+tiempoTarea){
        fac = 1;
    }
    if(tiempoMaq[j]+tiempos[j][indices[j]].tiempo ==
tiempoMaq[maq]+tiempoTarea){
        fac = 2;
    }
    return fac;
}

void avanceRapido(vector<vector<Tarea>> tiempos, int N, int M){

    //vector para tiempo (total actual) de las máquinas
    vector<int> tiempoMaq(M);
    //vector booleano para los usos de las tareas, false -> no usado
    vector<bool> candidatos(N);
    //vector de struct nuevo para guardar la solución, nº tarea, nº
maquina
    vector<Solucion> solucion(N);
```

```

//vector que lleva los índices de cada máquina, por qué hueco va de
cada máquina
vector<int> indices(M);

for(int i = 0; i < N; i++){
    float tiempoTarea = INFINITY;
    int maq = 0;
    int tarea = i;
    for(int j = 0; j < M; j++){
        while(indices[j] < N &&
candidatos[tiempos[j][indices[j]].tarea-1]){
            indices[j]++;
        }

        if(!candidatos[tiempos[j][indices[j]].tarea-1]){
            int fac =
esFactible(tiempoMaq,indices,j,maq,tiempoTarea,tiempos);
            if(fac == 1){
                tiempoTarea = tiempos[j][indices[j]].tiempo;

                maq = j;

                tarea = tiempos[j][indices[j]].tarea-1;

            }
            else if(fac == 2){
                if(tiempoMaq[j]<=tiempoMaq[maq]){
                    tiempoTarea = tiempos[j][indices[j]].tiempo;
                    maq = j;
                    tarea = tiempos[j][indices[j]].tarea-1;
                }
            }
        }
    }
    // guardar datos "finales"
    candidatos[tarea] = true;
    tiempoMaq[maq] += tiempoTarea;
    Solucion sol;
    sol.maquina = maq;
    sol.tarea = tarea;
    solucion[i] = sol;
    indices[maq]++;
}

// imprimir
int tiempoTotal = -1;

for(int i = 0; i < M; i++){

```

```

        if(tiempoMaq[i] > tiempoTotal){
            tiempoTotal = tiempoMaq[i];
        }
    }

    cout << tiempoTotal << endl;

    for(int i = 0; i < N; i++){
        cout << solucion[i].tarea+1 << " ";
    }
    cout << endl;
    for(int i = 0; i < N; i++){
        cout << solucion[i].maquina+1 << " ";
    }
    cout << endl;
    return;
}

int main()
{
    //Variables
    int P,N,M; //P -> número de casos; N -> número de tareas; M -> número
de máquinas
    cin >> P;
    while(P>0){
        cin >> N >> M;
        //podemos considerar que los tiempos que se van a guardar son los
pesos de las opciones
        vector<vector<Tarea>> tiempos(M,vector<Tarea>(N)); //filas
columnas
        for(int i = 0; i < N; i++){
            for(int j = 0; j < M; j++){
                cin >> tiempos[j][i].tiempo; //tiempo que tarda la tarea
en la máquina
                tiempos[j][i].tarea = i+1; //meter el dato leído en la
tabla
            }
        }
        for(int i = 0; i < M; i++){
            sort(tiempos[i].begin(),tiempos[i].end(),comparador);
        }

        avanceRapido(tiempos, N, M);
        P--;
    }

    return 0;
}

```

c. Número de envío

El número del envío válido es: 438.

d. Estudio teórico

Comparador $t(n) = 1 \rightarrow \boxed{\Theta(1)}$

establece $t_m(n) = 1 + 1 + 1 + 1 = 4 \rightarrow \boxed{\Theta(4)}$
 $t_H(n) = 1 + 2 + 2 + 1 = 6 \rightarrow \boxed{\Theta(6)}$

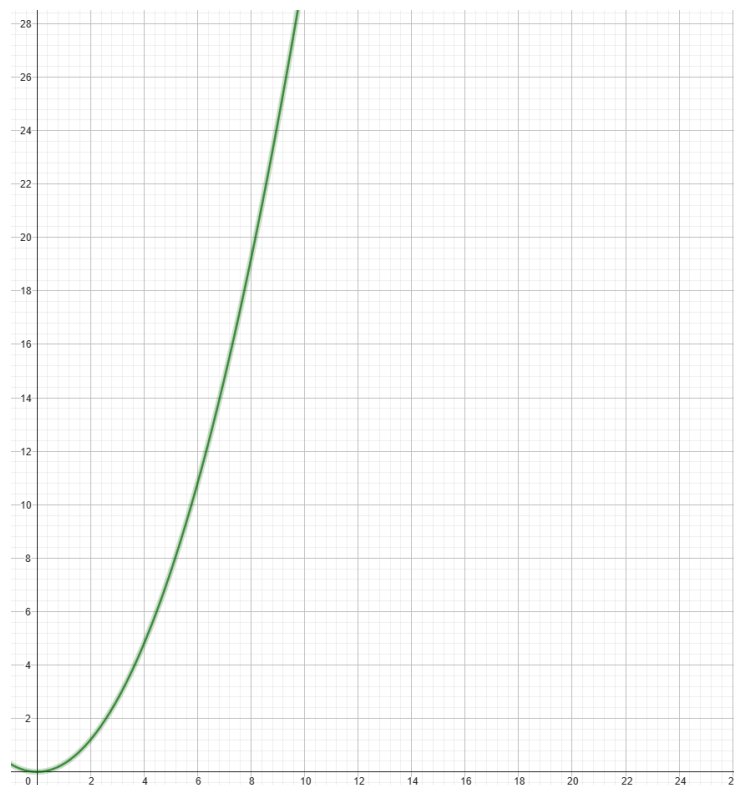
avance Rápido Nosotras hemos optado por ordenar en el main las tareas así que nuestro tiempo va a ser el del mejor caso siempre; pero, el trozo de código de ordenar las tareas es de $O(n \log n)$ así que lo vamos a sumar

$$t_m(n) = n \log n + 4 + \sum_{i=1}^N (4 + \sum_{j=1}^M (\sum_{k=j}^N 2 + 8)) = n \log n + 4 + \sum_{i=1}^N (4 + 2NM - 2M^2 + 10M) =$$

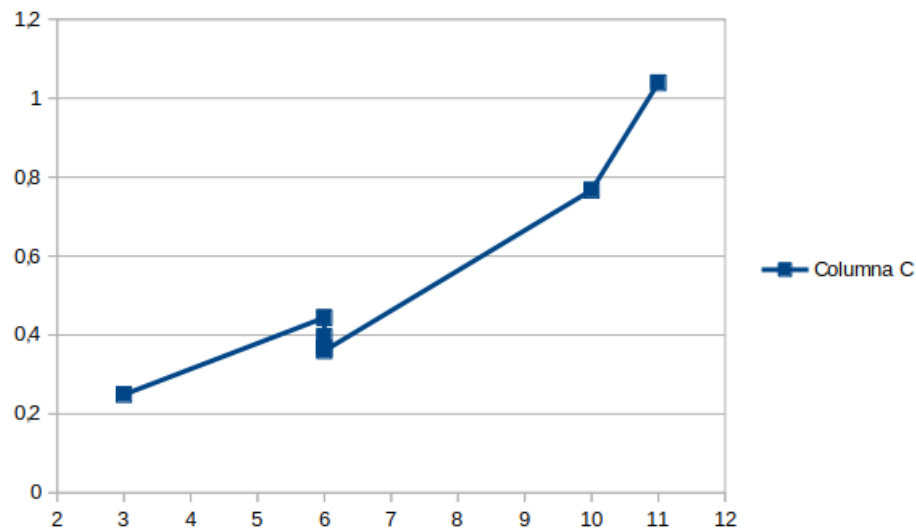
$$= N \log N + 4 + 4N + 2N^2M - 2M^2N + 10MN \rightarrow \boxed{\Omega(N^2M)}$$

$\downarrow$
 $2(N-j+1)$
 $2N - 2j + 2$

Si N y M valen lo mismo, el orden con el que nos quedamos es: $\Omega(N^2)$



e. Estudio experimental



f. Generador de casos

Como entrada para el estudio empírico hemos introducido casos en los que hemos fijado la N y hemos ido aumentando de uno en uno el tamaño de la M, ya que no hay caso mejor ni peor, sino que simplemente hay un caso promedio.

g. Contraste de estudio teórico y experimental

Podemos observar que no se parecen demasiado las gráficas, ya que es complicado que sepamos exactamente lo que hará el algoritmo. Seguramente, si tuviésemos una n mucho más grande, conseguiríamos una gráfica más parecida a la del estudio teórico.

3. BACKTRAKING

a. Justificación del tipo de árbol

Hemos decidido usar un árbol 6-ario (del tipo k-ario), debido a que cada uno de los empleados debe poder optar a quedarse con cualquier camiseta, por lo que optan a 6 opciones en un primer momento. Luego, con la poda vamos reduciendo el tamaño del árbol, pero debemos poder darles la opción.

b. Pseudocódigo

(Tratamos a los voluntarios como empleados)

Constantes:

- NUM_TALLAS: se trata del número de tallas que encontramos, la usamos para calcular el número de camisetas por talla que tenemos. Son 6 porque tenemos XS, S, M, L, XL y XXL.
- Las siguientes constantes establecen el “número de nodo del árbol” que representa cada talla.

Tenemos dos estructuras de datos diferentes. Una representa las Camisetas, contiene su talla y la cantidad de ellas que hay. La otra representa al Empleado, contiene las dos tallas que puede usar.

// Definición de constantes

```
const NUM_TALLAS = 6
```

```
const XS = 0
```

```
const S = 1
```

```
const M = 2
```

```
const L = 3
```

```
const XL = 4
```

```
const XXL = 5
```

// Definición de estructuras

```
struct Camisetas {
```

```
    cadena talla
```

```
    entero cantidad
```

```
}
```

```
struct Empleado {
```

```
    cadena talla1
```

```
    cadena talla2
```

```
}
```


Esta función genera el hermano contiguo del nodo actual si es que hay.

// Función para generar la siguiente solución

```
función generar(entero nivel, array sol[]) → entero {  
    devolver sol[nivel] + 1 // Avanza al siguiente hermano  
}
```

Esta función decide si se puede continuar con la ejecución por el nodo en el que estamos a partir de comprobar si las restricciones se cumplen o no. Estas son:

- Que todavía queden camisetas de la talla que estamos intentando tomar.
- Que le sirva al empleado actual.
- Que no nos hayamos pasado de la altura máxima del árbol, es decir, que no nos hayamos pasado del número de empleados que hay, representado por los niveles del árbol.

// Función que verifica el criterio de asignación de camisetas para un empleado específico

```
función criterio(entero nivel, array sol[], array empleados[]) → booleano {  
    booleano seguir = falso  
    cadena camisetaActual = camisetas[sol[nivel]].talla  
    si (camisetas[sol[nivel]].cantidad >= 0 && (empleados[nivel].talla1 == camisetaActual ||  
empleados[nivel].talla2 == camisetaActual) && nivel < m) entonces  
        seguir = verdadero  
    devolver seguir  
}
```

Esta función comprueba si se ha conseguido llegar a una solución.

- tieneSol indica si el empleado tiene una camiseta asignada o no.
- esSol indica si se ha encontrado la solución o no.
- criterioVal indica si se cumple el criterio o no.

Se ejecuta un bucle para comprobar si todos los empleados tienen una camiseta asignada.

Se ejecuta otro bucle que comprueba que se cumpla el criterio para todos los empleados.

Por último se comprueba si se ha llegado al último nivel del árbol y los dos booleanos con las soluciones de los bucles anteriores.

// Función que verifica si se alcanzó una solución válida

```
función solucion(entero nivel, array sol[], array empleados[]) → booleano {  
    booleano tieneSol = verdadero  
    booleano esSol = falso  
    booleano criterioVal = verdadero  
    para cada i en [0, m) hacer // Verifica que todos los empleados tengan una camiseta asignada  
        si (sol[i] == -1) entonces  
            tieneSol = falso  
    para cada i en [0, nivel] hacer  
        si (!criterio(i, sol, empleados)) entonces  
            criterioVal = falso  
    si (nivel == m - 1 && tieneSol && criterioVal) entonces // Si está en el último nivel, en el  
        último empleado y todos los empleados tienen camiseta  
            esSol = verdadero  
    devolver esSol  
}
```

Esta función comprueba que el nodo actual tenga más hermanos.

// Función que verifica si hay más hermanos posibles para explorar

```
función masHermanos(entero nivel, array sol[]) → booleano {  
    booleano mas = Falso  
    si (sol[nivel] < NUM_TALLAS) entonces // Comprueba que se pueda seguir con más  
        hermanos por la derecha  
        mas = Verdadero  
    devolver mas  
}
```

Esta función se ocupa de devolver una camiseta que se había repartido ya.

// Función para devolver una camiseta a la reserva

procedimiento devolverCamiseta(Entero nivel, Arreglo sol[]) {

 según sol[nivel] // Devuelve la camiseta

 caso XS:

 camisetas[XS].cantidad++

 romper

 caso S:

 camisetas[S].cantidad++

 romper

 caso M:

 camisetas[M].cantidad++

 romper

 caso L:

 camisetas[L].cantidad++

 romper

 caso XL:

 camisetas[XL].cantidad++

 romper

 caso XXL:

 camisetas[XXL].cantidad++

 romper

 porDefecto:

 romper

}

Esta función se encarga de imprimir la solución.

// Función para imprimir la respuesta

```
procedimiento imprimir(entero fin, entero nivel) {  
    si (nivel == -1 || !fin) entonces  
        escribir "NO"  
    sino  
        escribir "YES"  
}
```

Esta función contiene el algoritmo principal de backtracking.

- “nivel” indica la altura del árbol a la que estamos.
- El array “sol[m]” contiene la camiseta que se le asigna a cada empleado.

Se realiza un bucle para inicializar el array “sol” a -1, indicando que ningún empleado tiene asignada una camiseta.

- “fin” indica si hemos encontrado la solución.

El siguiente bucle usa en cada iteración las funciones de las anteriores:

- Genera el nodo a comprobar y asigna la camiseta correspondiente a ese nodo al empleado que estamos tratando.
- Se comprueba, si estamos tratando el último empleado, es decir, si estamos en el último nivel; si hemos encontrado la solución. En caso afirmativo pone fin a true.
- Se comprueba si se cumplen todavía las restricciones. En caso afirmativo se avanza en el árbol hasta el siguiente nivel.
- Si nada de lo anterior se cumple, quiere decir que hay que retroceder, ya que por ahí no estaría la solución y necesitamos hacer podar el árbol. Mientras que no lleguemos al principio del árbol y el nodo que se comprueba no tenga hermanos por los que seguir, se devuelve la camiseta del nodo que abandonamos, el empleado ya no tiene camiseta asignada y vamos a un nivel anterior. Si no nos hemos pasado del principio del árbol, se devuelve también la camiseta del empleado, el nodo, que dejamos para pasar a un hermano o terminar la ejecución.

Una vez termine el bucle, imprimirá la solución.

// Función principal de backtracking

```
procedimiento backTracking(array empleados[], entero n, entero m) {  
    entero nivel = 0  
    array sol[m]  
    para cada i en [0, m) hacer  
        sol[i] = -1  
    booleano fin = falso
```

```

repetir mientras (!fin y nivel != -1)
    sol[nivel] = generar(nivel, sol)
    si (sol[nivel] < NUM_TALLAS) entonces
        según sol[nivel]
            caso XS:
                camisetas[XS].cantidad--
                romper
            caso S:
                camisetas[S].cantidad--
                romper
            caso M:
                camisetas[M].cantidad--
                romper
            caso L:
                camisetas[L].cantidad--
                romper
            caso XL:
                camisetas[XL].cantidad--
                romper
            caso XXL:
                camisetas[XXL].cantidad--
                romper
            porDefecto:
                romper
        si (nivel == m - 1 && solucion(nivel, sol, empleados)) entonces
            fin = Verdadero
        sino si (criterio(nivel, sol, empleados)) entonces
            nivel++
        sino
            mientras (nivel >= 0 y !masHermanos(nivel, sol)) hacer
                devolverCamiseta(nivel, sol)
            sol[nivel] = -1

```

```

        nivel--
    si (nivel >= 0) entonces
        devolverCamiseta(nivel, sol)
    imprimir(fin, nivel)
}

```

El main se ocupa de leer el número de casos de prueba, el número total de camisetas y el número de empleados. Establece cuántas camisetas de cada talla tenemos e inicializa el array “camisetas”, definido como variable global junto con “n” y “m”. Después inicializa el array “empleados” con las tallas que puede usar cada empleado. Llama a backtracking para buscar la solución.

// Función principal

```

procedimiento Principal() {
    entero P // P → número de casos de prueba; N → múltiplo de 6 entre 1 y 36, número de
    camisetas; M → número de voluntarios; N >= M

    leer P

    mientras (P > 0) hacer
        leer n, m

        // Inicialización del array de camisetas
        entero tallas = n / NUM_TALLAS
        camisetas[XS].talla = "XS"
        camisetas[S].talla = "S"
        camisetas[M].talla = "M"
        camisetas[L].talla = "L"
        camisetas[XL].talla = "XL"
        camisetas[XXL].talla = "XXL"
        para cada i en [0, n) hacer
            camisetas[i].cantidad = tallas

        // Lectura de los empleados
        Empleado empleados[m]
        para cada i en [0, m) hacer
            leer empleados[i].talla1, empleados[i].talla2

        // Llamada a la función de backtracking
        backTracking(empleados, n, m)

    P--
}

```

c. Código

```
#include <iostream>

using namespace std;

struct Camisetas {
    string talla;
    int cantidad;
};
struct Empleado {
    string talla1;
    string talla2;
};

const int NUM_TALLAS = 6;
const int XS = 0;
const int S = 1;
const int M = 2;
const int L = 3;
const int XL = 4;
const int XXL = 5;

int n;
int m;

Camisetas camisetas[NUM_TALLAS]; //array para las camisetas que quedan

int generar(int nivel, int sol[]){
    return sol[nivel]+1; //avanza al siguiente hermano
}

bool criterio (int nivel, int sol[], Empleado empleados[]){
    bool seguir = false;
    string camisetaActual = camisetas[sol[nivel]].talla;
    if(camisetas[sol[nivel]].cantidad >= 0 && (empleados[nivel].talla1 ==
camisetaActual || empleados[nivel].talla2 == camisetaActual) && nivel <
m) {

        seguir = true;
    }

    return seguir;
}

bool solucion(int nivel, int sol[], Empleado empleados[]){
    bool tieneSol = true;
    bool esSol = false;
    bool criterioVal = true;
```

```

        //bool camisetasRep = true;
        for(int i = 0; i < m; i++){ //busca que algún empleado no tenga
camiseta todavía
            if(sol[i] == -1){
                tieneSol = false;
            }
        }
        for(int i = 0; i <= nivel; i++){
            if(!criterio(i,sol,empleados)){
                criterioVal = false;
            }
        }
        if(nivel == m-1 && tieneSol && criterioVal){ //si está en el último
nivel, en el último empleado y todos los empleados tienen camiseta
            esSol = true;
        }
        return esSol;
    }

bool masHermanos(int nivel, int sol[]){
    bool mas = false;
    if(sol[nivel] < NUM_TALLAS){ //comprueba que se pueda seguir con más
hermanos por la derecha
        mas = true;
    }
    return mas;
}

void devolverCamiseta(int nivel, int sol[]){
    switch (sol[nivel]){ //devuelve la camiseta
        case XS:
            camisetas[XS].cantidad++;
            break;
        case S:
            camisetas[S].cantidad++;
            break;
        case M:
            camisetas[M].cantidad++;
            break;
        case L:
            camisetas[L].cantidad++;
            break;
        case XL:
            camisetas[XL].cantidad++;
            break;
        case XXL:
            camisetas[XXL].cantidad++;
            break;
        default:
    
```



```

        break;
    }
}

void imprimir(int fin, int nivel){
    if(nivel == -1 || !fin){
        cout << "NO" << endl;
    }
    else {
        cout << "YES" << endl;
    }
}

void backTracking(Empleado empleados[], int n, int m){
    int nivel = 0;
    int sol[m];
    for(int i = 0; i < m; i++){
        sol[i] = -1;
    }
    bool fin = false;
    do{
        sol[nivel] = generar(nivel,sol);
        if(sol[nivel] < NUM_TALLAS){
            switch (sol[nivel]){
                case XS:
                    camisetas[XS].cantidad--;
                    break;
                case S:
                    camisetas[S].cantidad--;
                    break;
                case M:
                    camisetas[M].cantidad--;
                    break;
                case L:
                    camisetas[L].cantidad--;
                    break;
                case XL:
                    camisetas[XL].cantidad--;
                    break;
                case XXL:
                    camisetas[XXL].cantidad--;
                    break;
                default:
                    break;
            }
        }
        if(nivel==m-1 && solucion(nivel,sol,empleados)){
            fin = true;
        }
    }
}

```

```

        else if(criterio(nivel,sol,empleados)){
            nivel++;
        }
        else {
            while(nivel>=0 && !masHermanos(nivel, sol)){
                devolverCamiseta(nivel,sol);
                sol[nivel]=-1;
                nivel--;
            }
            if(nivel>=0){
                devolverCamiseta(nivel,sol);
            }
        }
    }while(!fin && nivel!=-1);

    imprimir(fin, nivel);
}

int main(){
    int P; //P -> número de casos de prueba; N -> múltiplo de 6 entre 1 y
36, número de camisetas; M -> número de voluntarios; N >= M
    cin >> P;
    while(P > 0){
        cin >> n >> m;
        // inicialización array
        int tallas = n/NUM_TALLAS;
        camisetas[XS].talla = "XS";
        camisetas[S].talla = "S";
        camisetas[M].talla = "M";
        camisetas[L].talla = "L";
        camisetas[XL].talla = "XL";
        camisetas[XXL].talla = "XXL";
        for(int i = 0; i < n; i++){
            camisetas[i].cantidad = tallas;
        }
        Empleado empleados[m];
        for(int i = 0; i < m; i++){
            cin >> empleados[i].talla1 >> empleados[i].talla2;
        }

        backTracking(empleados, n, m);
        P--;
    }
    return 0;
}

```

d. Numero de envío

El número del envío válido es: 434.

e. Estudio teórico

generar $t(m) = 1 \rightarrow \boxed{O(1)}$

criterio $t_m(m) = 2 + 1 + 1 = 4 \rightarrow \boxed{O(4)}$
 $t_M(m) = 3 + 2 + 1 = 6 \rightarrow \boxed{O(6)}$

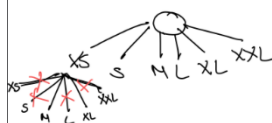
solucion $t_m(m) = 3 + \sum_{i=1}^m (2) + \sum_{i=2}^{m+1} (2) + 1 + 1 + 1 = 7 + 4m \rightarrow \boxed{O(m)}$
 $t_M(m) = 3 + \sum_{i=1}^{m-1} (3) + \sum_{i=1}^{m+1} (3) + 1 + 2 + 1 = 8 + 6m \rightarrow \boxed{O(m)}$

mas hermanos $t_m(m) = 1 + 1 + 1 = 3 \rightarrow \boxed{O(3)}$
 $t_M(m) = 1 + 2 + 1 = 4 \rightarrow \boxed{O(4)}$

devolver Camiseta $t(m) = 3 \rightarrow \boxed{O(3)}$

imprimir $t(m) = 2 \rightarrow \boxed{O(2)}$

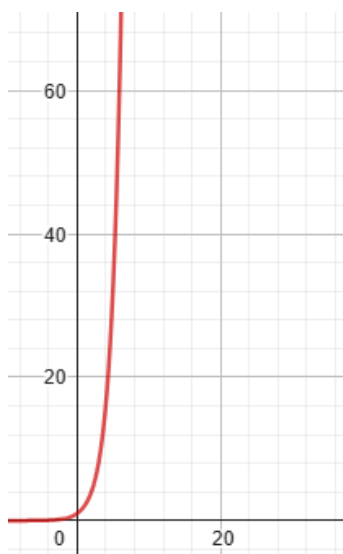
backtracking



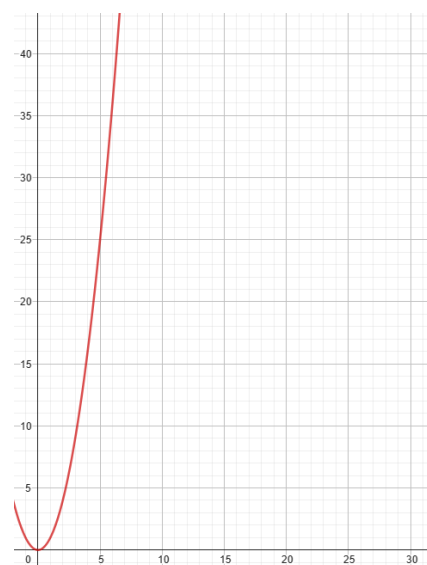
Es muy difícil saber con seguridad que es lo que hará el código. Lo único que tenemos seguro es que cuando empieza tenemos un árbol k -ario donde $k=6$, que son las tallas que tenemos disponibles. Si no tuviéramos la restricción del criterio tendríamos que recorrer el árbol entero llegando a $O(6^m)$, pero con la restricción de cada voluntario nos quedamos que el orden es (2^m) .
 m para nosotras es el nº de voluntario y el nivel del árbol.

Tomamos que el peor caso es, entonces 2^m y que el mejor caso es m^2 .

Peor caso:

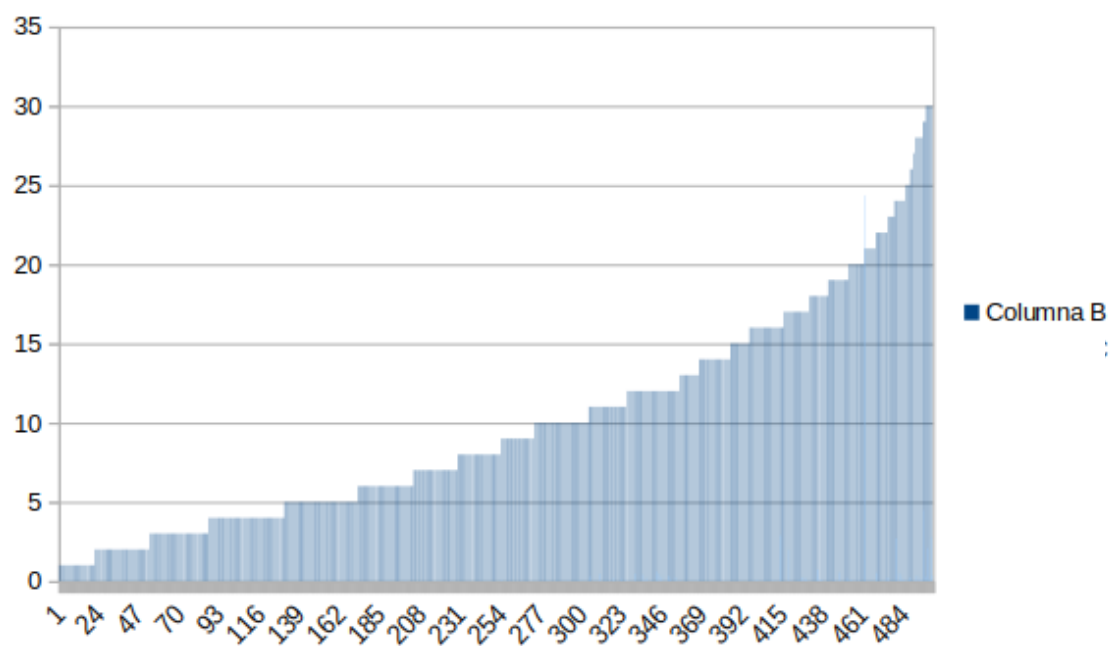


Mejor caso:

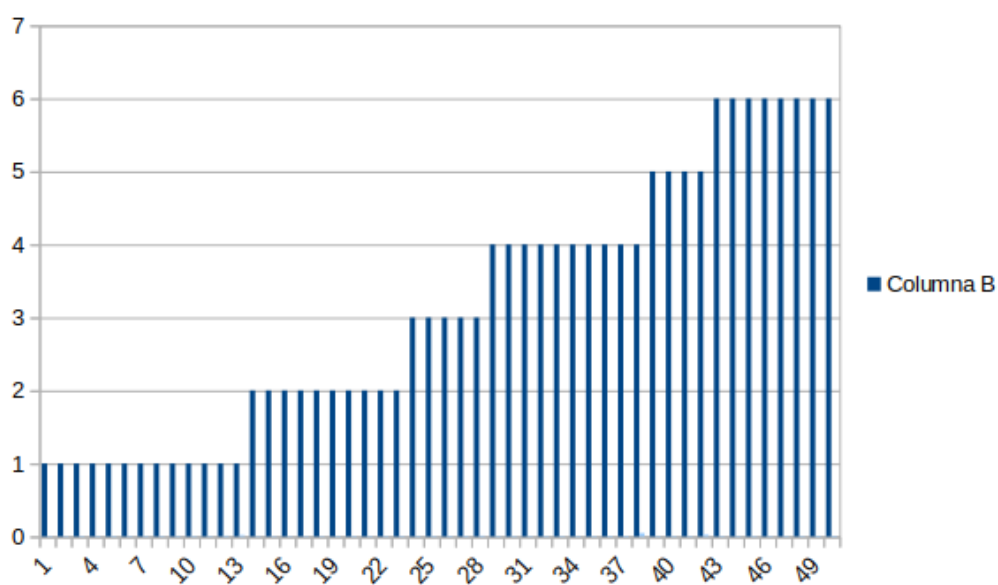


f. Estudio experimental

Peor caso:



Mejor caso:



g. Generador de casos

Hemos creado un generador para el mejor caso.

El mejor caso se podría decir que es que todos los empleados o voluntarios puedan usar camisetas con talla “XS” y que haya tantas camisetas de esa talla como empleados. Así, hemos hecho que el número de camisetas total y, por lo tanto, el número de camisetas de cada talla, dependa del número de empleados. Este ha tenido que ser como máximo 6 debido a que ni N ni M pueden ser mayor que 36. Como hay 6 tallas diferentes, podemos tener, como mucho, 6 camisetas de cada talla ($6 \cdot 6 = 36$).

```
//#include <iostream>
#include <fstream>
//#include <sys/time.h>
//#include <sstream>

using namespace std;

ofstream casos;

void generadorMejorCaso(){
    casos.open("casos.txt");
    int p = rand() % 50 + 1;
    casos << p;
    casos << "\n";
    for (int i = 0; i < p; i++){
        int m = rand() % 6 + 1;
        int n = m * 6;
        casos << n;
        casos << " ";
        casos << m;
        casos << "\n";
        for(int j = 0; j < m; j++){
            casos << "XS";
            casos << " ";
            casos << "S";
            casos << "\n";
        }
    }
}

int main() {
    srand(time(NULL));
    generadorMejorCaso();
    return 0;
}
```

h. Contraste de estudio teórico y experimental

Si tuviésemos un tamaño mayor de M , podríamos observar con más claridad el parecido entre el supuesto resultado del peor caso teórico y el experimental.

En el caso del supuesto mejor caso, podríamos decir que en cierto modo se parece, seguramente, con mayor tamaño de M , también se podría observar mejor.

4. Conclusiones

Ha sido un trabajo que nos ha refrescado la memoria acerca del tratamiento de árboles y que nos ha ayudado a una mejor comprensión de estos. Además, hemos aprendido a realizar dos métodos de búsqueda de soluciones diferentes que no habíamos tratado previamente. Consideramos que hemos aprendido bastante sobre estos temas.

Hemos tardado en realizar este trabajo alrededor de 15 horas.